

Chapter 8. Sharing Functions with Code Libraries

By now you should realize that the computer has to do a lot of work even for simple tasks. Because of that, you have to do a lot of work to write the code for a computer to even do simple tasks. In addition, programming tasks are usually not very simple. Therefore, we need a way to make this process easier on ourselves. There are several ways to do this, including:

- Write code in a high-level language instead of assembly language
- Have lots of pre-written code that you can cut and paste into your own programs
- Have a set of functions on the system that are shared among any program that wishes to use it

All three of these are usually used to some degree in any given project. The first option will be explored further in Chapter 11. The second option is useful but it suffers from some drawbacks, including:

- Code that is copied often has to be majorly modified to fit the surrounding code.
- Every program containing the copied code has the same code in it, thus wasting a lot of space.
- If a bug is found in any of the copied code it has to be fixed in every application program.

Therefore, the second option is usually used sparingly. It is usually only used in cases where you copy and paste skeleton code for a specific type of task, and add in your program-specific details. The third option is the one that is used the most often. The third option includes having a central repository of shared code. Then, instead of each program wasting space storing the same copies of functions, they can simply point to the shared libraries which contain the functions they need. If a

Chapter 8. Sharing Functions with Code Libraries

bug is found in one of these functions, it only has to be fixed within the single function library file, and all applications which use it are automatically updated. The main drawback with this approach is that it creates some dependency problems, including:

- If multiple applications are all using the shared file, how do we know when it is safe to delete the file? For example, if three applications are sharing a file of functions and 2 of the programs are deleted, how does the system know that there still exists an application that uses that code, and therefore it shouldn't be deleted?
- Some programs inadvertently rely on bugs within shared functions. Therefore, if upgrading the shared program fixes a bug that a program depended on, it could cause that application to cease functioning.

These problems are what lead to what is known as "DLL hell". However, it is generally assumed that the advantages outweigh the disadvantages.

In programming, these shared code files are referred to as *shared libraries*, *shared objects*, *dynamic-link libraries*, *DLLs*, or *.so files*. We will refer to them as *shared libraries*.

Using a Shared Library

The program we will examine here is simple - it writes the characters `hello world` to the screen and exits. The regular program, `helloworld-nolib.s`, looks like this:

```
#PURPOSE:  This program writes the message "hello world" and
#           exits
#
#include "linux.s"
```

```
.section .data

helloworld:
    .ascii "hello world\n"
helloworld_end:

.equ helloworld_len, helloworld_end - helloworld

.section .text
.globl _start
_start:
    movl $STDOUT, %ebx
    movl $helloworld, %ecx
    movl $helloworld_len, %edx
    movl $SYS_WRITE, %eax
    int $LINUX_SYSCALL

    movl $0, %ebx
    movl $SYS_EXIT, %eax
    int $LINUX_SYSCALL
```

That's not too long. However, take a look at how short `helloworld-lib` is which uses a library:

```
#PURPOSE:  This program writes the message "hello world" and
#           exits
#

.section .data

helloworld:
```

Chapter 8. Sharing Functions with Code Libraries

```
.ascii "hello world\n\0"

.section .text
.globl _start
_start:
    pushl $helloworld
    call printf

    pushl $0
    call exit
```

It's even shorter!

Now, building programs which use shared libraries is a little different than normal. You can build the first program normally by doing this:

```
as helloworld-nolib.s -o helloworld-nolib.o
ld helloworld-nolib.o -o helloworld-nolib
```

However, in order to build the second program, you have to do this:

```
as helloworld-lib.s -o helloworld-lib.o
ld -dynamic-linker /lib/ld-linux.so.2 \
    -o helloworld-lib helloworld-lib.o -lc
```

Remember, the backslash in the first line simply means that the command continues on the next line. The option `-dynamic-linker /lib/ld-linux.so.2` allows our program to be linked to libraries. This builds the executable so that before executing, the operating system will load the program `/lib/ld-linux.so.2` to load in external libraries and link them with the program. This program is known as a *dynamic linker*.

The `-lc` option says to link to the `c` library, named `libc.so` on GNU/Linux systems. Given a library name, `c` in this case (usually library names are longer than a single letter), the GNU/Linux linker prepends the string `lib` to the beginning of

the library name and appends `.so` to the end of it to form the library's filename. This library contains many functions to automate all types of tasks. The two we are using are `printf`, which prints strings, and `exit`, which exits the program.

Notice that the symbols `printf` and `exit` are simply referred to by name within the program. In previous chapters, the linker would resolve all of the names to physical memory addresses, and the names would be thrown away. When using dynamic linking, the name itself resides within the executable, and is resolved by the dynamic linker when it is run. When the program is run by the user, the dynamic linker loads the shared libraries listed in our link statement, and then finds all of the function and variable names that were named by our program but not found at link time, and matches them up with corresponding entries in the shared libraries it loads. It then replaces all of the names with the addresses which they are loaded at. This sounds time-consuming. It is to a small degree, but it only happens once - at program startup time.

How Shared Libraries Work

In our first programs, all of the code was contained within the source file. Such programs are called *statically-linked executables*, because they contained all of the necessary functionality for the program that wasn't handled by the kernel. In the programs we wrote in Chapter 6, we used both our main program file and files containing routines used by multiple programs. In these cases, we combined all of the code together using the linker at link-time, so it was still statically-linked. However, in the `helloworld-lib` program, we started using shared libraries. When you use shared libraries, your program is then *dynamically-linked*, which means that not all of the code needed to run the program is actually contained within the program file itself, but in external libraries.

When we put the `-lc` on the command to link the `helloworld` program, it told the linker to use the `c` library (`libc.so`) to look up any symbols that weren't already defined in `helloworld.o`. However, it doesn't actually add any code to

Chapter 8. Sharing Functions with Code Libraries

our program, it just notes in the program where to look. When the `helloworld` program begins, the file `/lib/ld-linux.so.2` is loaded first. This is the dynamic linker. This looks at our `helloworld` program and sees that it needs the `c` library to run. So, it searches for a file called `libc.so` in the standard places (listed in `/etc/ld.so.conf` and in the contents of the `LD_LIBRARY_PATH` environment variable), then looks in it for all the needed symbols (`printf` and `exit` in this case), and then loads the library into the program's virtual memory. Finally, it replaces all instances of `printf` in the program with the actual location of `printf` in the library.

Run the following command:

```
ldd ./helloworld-nolib
```

It should report back `not a dynamic executable`. This is just like we said - `helloworld-nolib` is a statically-linked executable. However, try this:

```
ldd ./helloworld-lib
```

It will report back something like

```
libc.so.6 => /lib/libc.so.6 (0x4001d000)
/lib/ld-linux.so.2 => /lib/ld-linux.so.2 (0x40000000)
```

The numbers in parenthesis may be different on your system. This means that the program `helloworld` is linked to `libc.so.6` (the `.6` is the version number), which is found at `/lib/libc.so.6`, and `/lib/ld-linux.so.2` is found at `/lib/ld-linux.so.2`. These libraries have to be loaded before the program can be run. If you are interested, run the `ldd` program on various programs that are on your Linux distribution, and see what libraries they rely on.

Finding Information about Libraries

Okay, so now that you know about libraries, the question is, how do you find out

what libraries you have on your system and what they do? Well, let's skip that question for a minute and ask another question: How do programmers describe functions to each other in their documentation? Let's take a look at the function `printf`. It's calling interface (usually referred to as a *prototype*) looks like this:

```
int printf(char *string, ...);
```

In Linux, functions are described in the C programming language. In fact, most Linux programs are written in C. That is why most documentation and binary compatibility is defined using the C language. The interface to the `printf` function above is described using the C programming language.

This definition means that there is a function `printf`. The things inside the parenthesis are the function's parameters or arguments. The first parameter here is `char *string`. This means there is a parameter named `string` (the name isn't important, except to use for talking about it), which has a type `char *`. `char` means that it wants a single-byte character. The `*` after it means that it doesn't actually want a character as an argument, but instead it wants the address of a character or sequence of characters. If you look back at our `helloworld` program, you will notice that the function call looked like this:

```
pushl $hello  
call printf
```

So, we pushed the address of the `hello` string, rather than the actual characters. You might notice that we didn't push the length of the string. The way that `printf` found the end of the string was because we ended it with a null character (`\0`). Many functions work that way, especially C language functions. The `int` before the function definition tell what type of value the function will return in `%eax` when it returns. `printf` will return an `int` when it's through. Now, after the `char *string`, we have a series of periods, `...`. This means that it can take an indefinite number of additional arguments after the string. Most functions can only take a specified number of arguments. `printf`, however, can take many. It will look into the `string` parameter, and everywhere it sees the characters `%s`, it will

Chapter 8. Sharing Functions with Code Libraries

look for another string from the stack to insert, and everywhere it sees %d it will look for a number from the stack to insert. This is best described using an example:

```
#PURPOSE:  This program is to demonstrate how to call printf
#

.section .data

#This string is called the format string.  It's the first
#parameter, and printf uses it to find out how many parameters
#it was given, and what kind they are.
firststring:
.ascii "Hello! %s is a %s who loves the number %d\n\0"
name:
.ascii "Jonathan\0"
personstring:
.ascii "person\0"
#This could also have been an .equ, but we decided to give it
#a real memory location just for kicks
numberloved:
.long 3

.section .text
.globl _start
_start:
#note that the parameters are passed in the
#reverse order that they are listed in the
#function's prototype.
pushl numberloved    #This is the %d
pushl $personstring  #This is the second %s
pushl $name          #This is the first %s
pushl $firststring   #This is the format string
                    #in the prototype

call  printf
```



```
pushl $0
call  exit
```

Type it in with the filename `printf-example.s`, and then do the following commands:

```
as printf-example.s -o printf-example.o
ld printf-example.o -o printf-example -lc \
    -dynamic-linker /lib/ld-linux.so.2
```

Then run the program with `./printf-example`, and it should say this:

```
Hello! Jonathan is a person who loves the number 3
```

Now, if you look at the code, you'll see that we actually push the format string last, even though it's the first parameter listed. You always push a functions parameters in reverse order.¹ You may be wondering how the `printf` function knows how many parameters there are. Well, it searches through your string, and counts how many `%ds` and `%ss` it finds, and then grabs that number of parameters from the stack. If the parameter matches a `%d`, it treats it as a number, and if it matches a `%s`, it treats it as a pointer to a null-terminated string. `printf` has many more features than this, but these are the most-used ones. So, as you can see, `printf` can make output a lot easier, but it also has a lot of overhead, because it has to count the number of characters in the string, look through it for all of the control characters it needs to replace, pull them off the stack, convert them to a

1. The reason that parameters are pushed in the reverse order is because of functions which take a variable number of parameters like `printf`. The parameters pushed in last will be in a known position relative to the top of the stack. The program can then use these parameters to determine where on the stack the additional arguments are, and what type they are. For example, `printf` uses the format string to determine how many other parameters are being sent. If we pushed the known arguments first, you wouldn't be able to tell where they were on the stack.

Chapter 8. Sharing Functions with Code Libraries

suitable representation (numbers have to be converted to strings, etc), and stick them all together appropriately.

We've seen how to use the C programming language prototypes to call library functions. To use them effectively, however, you need to know several more of the possible data types for reading functions. Here are the main ones:

`int`

An `int` is an integer number (4 bytes on x86 processor).

`long`

A `long` is also an integer number (4 bytes on an x86 processor).

`long long`

A `long long` is an integer number that's larger than a `long` (8 bytes on an x86 processor).

`short`

A `short` is an integer number that's shorter than an `int` (2 bytes on an x86 processor).

`char`

A `char` is a single-byte integer number. This is mostly used for storing character data, since ASCII strings usually are represented with one byte per character.

`float`

A `float` is a floating-point number (4 bytes on an x86 processor). Floating-point numbers will be explained in more depth in the Section called *Floating-point Numbers* in Chapter 10.

`double`

A `double` is a floating-point number that is larger than a `float` (8 bytes on an x86 processor).

`unsigned`

`unsigned` is a modifier used for any of the above types which keeps them from being used as signed quantities. The difference between signed and unsigned numbers will be discussed in Chapter 10.

`*`

An asterisk (`*`) is used to denote that the data isn't an actual value, but instead is a pointer to a location holding the given value (4 bytes on an x86 processor). So, let's say in memory location `my_location` you have the number 20 stored. If the prototype said to pass an `int`, you would use direct addressing mode and do `pushl my_location`. However, if the prototype said to pass an `int *`, you would do `pushl $my_location` - an immediate mode push of the address that the value resides in. In addition to indicating the address of a single value, pointers can also be used to pass a sequence of consecutive locations, starting with the one pointed to by the given value. This is called an array.

`struct`

A `struct` is a set of data items that have been put together under a name. For example you could declare:

```
struct teststruct {  
    int a;  
    char *b;  
};
```

and any time you ran into `struct teststruct` you would know that it is actually two words right next to each other, the first being an integer, and the second a pointer to a character or group of characters. You never see structs

Chapter 8. Sharing Functions with Code Libraries

passed as arguments to functions. Instead, you usually see pointers to structs passed as arguments. This is because passing structs to functions is fairly complicated, since they can take up so many storage locations.

typedef

A typedef basically allows you to rename a type. For example, I can do `typedef int myowntype;` in a C program, and any time I typed `myowntype`, it would be just as if I typed `int`. This can get kind of annoying, because you have to look up what all of the typedefs and structs in a function prototype really mean. However, typedefs are useful for giving types more meaningful and descriptive names.

Compatibility Note: The listed sizes are for intel-compatible (x86) machines. Other machines will have different sizes. Also, even when parameters shorter than a word are passed to functions, they are passed as longs on the stack.

That's how to read function documentation. Now, let's get back to the question of how to find out about libraries. Most of your system libraries are in `/usr/lib` or `/lib`. If you want to just see what symbols they define, just run `objdump -R FILENAME` where `FILENAME` is the full path to the library. The output of that isn't too helpful, though, for finding an interface that you might need. Usually, you have to know what library you want at the beginning, and then just read the documentation. Most libraries have manuals or man pages for their functions. The web is the best source of documentation for libraries. Most libraries from the GNU project also have info pages on them, which are a little more thorough than man pages.

Useful Functions

Several useful functions you will want to be aware of from the c library include:

- `size_t strlen (const char *s)` calculates the size of null-terminated strings.
- `int strcmp (const char *s1, const char *s2)` compares two strings alphabetically.
- `char * strdup (const char *s)` takes the pointer to a string, and creates a new copy in a new location, and returns the new location.
- `FILE * fopen (const char *filename, const char *opentype)` opens a managed, buffered file (allows easier reading and writing than using file descriptors directly).²³
- `int fclose (FILE *stream)` closes a file opened with `fopen`.
- `char * fgets (char *s, int count, FILE *stream)` fetches a line of characters into string `s`.
- `int fputs (const char *s, FILE *stream)` writes a string to the given open file.
- `int fprintf (FILE *stream, const char *template, ...)` is just like `printf`, but it uses an open file rather than defaulting to using standard output.

You can find the complete manual on this library by going to

<http://www.gnu.org/software/libc/manual/>

2. `stdin`, `stdout`, and `stderr` (all lower case) can be used in these programs to refer to the files of their corresponding file descriptors.

3. `FILE` is a struct. You don't need to know it's contents to use it. You only have to store the pointer and pass it to the relevant other functions.

Building a Shared Library

Let's say that we wanted to take all of our shared code from Chapter 6 and build it into a shared library to use in our programs. The first thing we would do is assemble them like normal:

```
as write-record.s -o write-record.o
as read-record.s -o read-record.o
```

Now, instead of linking them into a program, we want to link them into a shared library. This changes our linker command to this:

```
ld -shared write-record.o read-record.o -o librecord.so
```

This links both of these files together into a shared library called `librecord.so`. This file can now be used for multiple programs. If we need to update the functions contained within it, we can just update this one file and not have to worry about which programs use it.

Let's look at how we would link against this library. To link the `write-records` program, we would do the following:

```
as write-records.s -o write-records
ld -L . -dynamic-linker /lib/ld-linux.so.2 \
    -o write-records -lrecord write-records.o
```

In this command, `-L .` told the linker to look for libraries in the current directory (it usually only searches `/lib` directory, `/usr/lib` directory, and a few others). As we've seen, the option `-dynamic-linker /lib/ld-linux.so.2` specified the dynamic linker. The option `-lrecord` tells the linker to search for functions in the file named `librecord.so`.

Now the `write-records` program is built, but it will not run. If we try it, we will get an error like the following:

```
./write-records: error while loading shared libraries:
```

```
librecord.so: cannot open shared object file: No such  
file or directory
```

This is because, by default, the dynamic linker only searches `/lib`, `/usr/lib`, and whatever directories are listed in `/etc/ld.so.conf` for libraries. In order to run the program, you either need to move the library to one of these directories, or execute the following command:

```
LD_LIBRARY_PATH=.  
export LD_LIBRARY_PATH
```

Alternatively, if that gives you an error, do this instead:

```
setenv LD_LIBRARY_PATH .
```

Now, you can run `write-records` normally by typing `./write-records`. Setting `LD_LIBRARY_PATH` tells the linker to add whatever paths you give it to the library search path for dynamic libraries

For further information about dynamic linking, see the following sources on the Internet:

- The man page for `ld.so` contains a lot of information about how the Linux dynamic linker works.
- <http://www.benyossef.com/presentations/dlink/> is a great presentation on dynamic linking in Linux.
- <http://www.linuxjournal.com/article.php?sid=1059> and <http://www.linuxjournal.com/article.php?sid=1060> provide a good introduction to the ELF file format, with more detail available at <http://www.cs.ucdavis.edu/~haungs/paper/node10.html>
- <http://www.iecc.com/linker/linker10.html> contains a great description of how dynamic linking works with ELF files.

Review

Know the Concepts

- What are the advantages and disadvantages of shared libraries?
- Given a library named 'foo', what would the library's filename be?
- What does the `ldd` command do?
- Let's say we had the files `foo.o` and `bar.o`, and you wanted to link them together, and dynamically link them to the library 'kramer'. What would the linking command be to generate the final executable?
- What is *typedef* for?
- What are *structs* for?
- What is the difference between a data element of type *int* and *int **? How would you access them differently in your program?
- If you had a object file called `foo.o`, what would be the command to create a shared library called 'bar'?
- What is the purpose of `LD_LIBRARY_PATH`?

Use the Concepts

- Rewrite one or more of the programs from the previous chapters to print their results to the screen using `printf` rather than returning the result as the exit status code. Also, make the exit status code be 0.
- Use the `factorial` function you developed in the Section called *Recursive Functions* in Chapter 4 to make a shared library. Then re-write the main program so that it links with the library dynamically.

- Rewrite the program above so that it also links with the 'c' library. Use the 'c' library's `printf` function to display the result of the `factorial` call.
- Rewrite the `toupper` program so that it uses the `c` library functions for files rather than system calls.

Going Further

- Make a list of all the environment variables used by the GNU/Linux dynamic linker.
- Research the different types of executable file formats in use today and in the history of computing. Tell the strengths and weaknesses of each.
- What kinds of programming are you interested in (graphics, databbases, science, etc.)? Find a library for working in that area, and write a program that makes some basic use of that library.
- Research the use of `LD_PRELOAD`. What is it used for? Try building a shared library that contained the `exit` function, and have it write a message to `STDERR` before exiting. Use `LD_PRELOAD` and run various programs with it. What are the results?

